

mdspan.at()

Document #: P3383R1
Date: 2024-11-19
Project: Programming Language C++
Audience: Library Evolution Working Group (LEWG)
SG23 Safety and Security
Reply-to: Stephan Lachnit
<stephan.lachnit@cern.ch>
Xavier Bonaventura (BMW)
<xavier.bonaventura@bmw.de>

Contents

- [1 Introduction](#)
- [2 Revision history](#)
 - [2.1 R1](#)
 - [2.2 R0](#)
- [3 Motivation](#)
 - [3.1 Safety](#)
 - [3.2 Consistency](#)
- [4 Impact On the Standard](#)
- [5 Wording](#)
- [6 Reference Implementation](#)
- [7 References](#)

§ 1 Introduction

This paper proposes element access with bounds checking to `std::mdspan` via `at()` member functions.

§ 2 Revision history

§ 2.1 R1

Update wording to align with [\[P2821R5\]](#):

- Bump `__cpp_lib_mdspan` instead of adding `__cpp_lib_mdspan_at`
- Mark `at()` as freestanding deleted and adjust freestanding comments of header and class

§ 2.2 R0

Initial version

§ 3 Motivation

§ 3.1 Safety

The new `at()` member functions provide memory-safe element access to `std::mdspan`, and thus have defined behavior. Out-of-bound access can be caught by catching the `std::out_of_range` exception.

§ 3.2 Consistency

In [P2821R5], element access with bounds checking via `at()` has been added to `std::span`. One of the main motivations for this change was consistency with other containers that have element access with bounds checking via `at()`. Similarly, such element access should be added to `std::mdspan`.

§ 4 Impact On the Standard

The impact of this proposal on the standard is low. The proposed function signatures for the `at()` member functions are identical to the function signatures for the subscript operators as proposed in [P0009R18].

One consideration is that the `at()` method has previously not been used with multi-dimensional arguments. However, this was also true for the subscript operator before the possibility was introduced in [P2128R6].

§ 5 Wording

The wording is relative to [N4993].

In 17.3.2 ([[version.syn](#)]), adjust the value of `__cpp_lib_mdspan` to the date of this proposal's adoption.

In 23.7.3.2 ([[mdspan.syn](#)]), change the following lines:

```
- // all freestanding
+ // mostly freestanding
namespace std {
```

```
// 23.7.3.6, class template mdspan
template<class ElementType, class Extents, class LayoutPolicy =
    layout_right,
    class AccessorPolicy = default_accessor<ElementType>>
- class mdspan;
+                                     class                               mdspan;
    // partially freestanding
```

In 23.7.3.6.1 ([\[mdspan.mdspan.overview\]](#)), add the following immediately after the subscript operators:

```
template<class... OtherIndexTypes>
    constexpr reference at(OtherIndexTypes... indices) const;    //
        freestanding-deleted
template<class OtherIndexType>
    constexpr reference at(span<OtherIndexType, rank()> indices)
        const;    // freestanding-deleted
template<class OtherIndexType>
    constexpr reference at(const array<OtherIndexType, rank()>&
        indices) const;    // freestanding-deleted
```

In 23.7.3.6.3 ([\[mdspan.mdspan.members\]](#)), add the following immediately after the subscript operators:

```
template<class... OtherIndexTypes>
    constexpr reference at(OtherIndexTypes... indices) const;
```

7 *Constraints:*

- (7.1) — `(is_convertible_v<OtherIndexTypes, index_type> && ...)` is true,
- (7.2) — `(is_nothrow_constructible_v<index_type, OtherIndexTypes> && ...)` is true, and
- (7.3) — `sizeof...(OtherIndexTypes) == rank()` is true.

8 *Returns:* `(*this)[indices...]`

- 9 *Throws:* `out_of_range` if
`indices_v[i] >= extent(i) || indices_v[i] < 0`
for any `indices_v[i]` in `vector<OtherIndexTypes>({indices...})`.

```
template<class OtherIndexType>
    constexpr reference at(span<OtherIndexType, rank()> indices)
        const;
```

```
template<class OtherIndexType>
    constexpr reference at(const array<OtherIndexType, rank()>&
        indices) const;
```

10 *Constraints:*

- (10.1) — `is_convertible_v<const OtherIndexType&, index_type>` is true, and

(10.2) — `is_nothrow_constructible_v<index_type, OtherIndexType>` is true. const

11 *Returns:* `(*this)[indices]`

12 *Throws:* `out_of_range` if `indices[i] >= extent(i) || indices[i] < 0` for any `indices[i]` in `indices`.

§ 6 Reference Implementation

The `at()` member functions have been implemented in the `std::mdspan` reference implementation from the Kokkos project at Sandia National Laboratories [[kokkos/mdspan](#)], see [[kokkos/mdspan#302](#)].

§ 7 References

[[kokkos/mdspan](#)] Reference implementation of `mdspan`.

<https://github.com/kokkos/mdspan>

[[kokkos/mdspan#302](#)] Add element access via `at()` to `std::mdspan`.

<https://github.com/kokkos/mdspan/pull/302>

[N4993] Thomas Köppe. 2024-10-16. Working Draft, Programming Languages — C++.

<https://wg21.link/n4993>

[P0009R18] Christian Trott, D.S. Hollman, Damien Lebrun-Grandie, Mark Hoemmen, Daniel Sunderland, H. Carter Edwards, Bryce Adelstein Lelbach, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Nevin Liber. 2022-07-13. MDSPAN.

<https://wg21.link/p0009r18>

[P2128R6] Corentin Jabot, Isabella Muerte, Daisy Hollman, Christian Trott, Mark Hoemmen. 2021-09-14. Multidimensional subscript operator.

<https://wg21.link/p2128r6>

[P2821R5] Jarrad J. Waterloo. 2023-12-18. `span.at()`.

<https://wg21.link/p2821r5>